this, you need to refer to the same behavior in a way other than `Behaviors.same`. To keep both values, you call `apply(current, max)` directly and pass the new updated values (keep in mind that the wallet has more logic than is shown here—this snippet shows only the relevant parts needed to hold the state of the actor: `total` and `max`):

```
object WalletState {

   def apply(total: Int, max: Int): Behavior[Command] =
        Behavior.receive { (context, message) =>          ❶
            ...
            val current = total + 1                       ❷
            ...
            apply(current, max)                           ❸
        }
}
```

❶ Input function

❷ Increases the total

❸ Output function designating the next behavior while keeping the new total and max

Because behaviors are functions, you can use their inputs and outputs to transfer state from one function to the next (from one behavior to the next). Here, by setting `apply(current, max)` at the end of `def apply(total: Int, max: Int)`, you handle the next message with the same apply function but with newly updated input parameters. The first time, `total` and `max` were set to `0` and `2` using `.apply(0,2)`—that is, `WalletState(0,2)`—thanks to Scala syntactic sugar. But after processing an `Increase` message, the next behavior will be `.apply(1,2)`. This way, the behavior has new input values, and the state is transferred from one message to the next.